

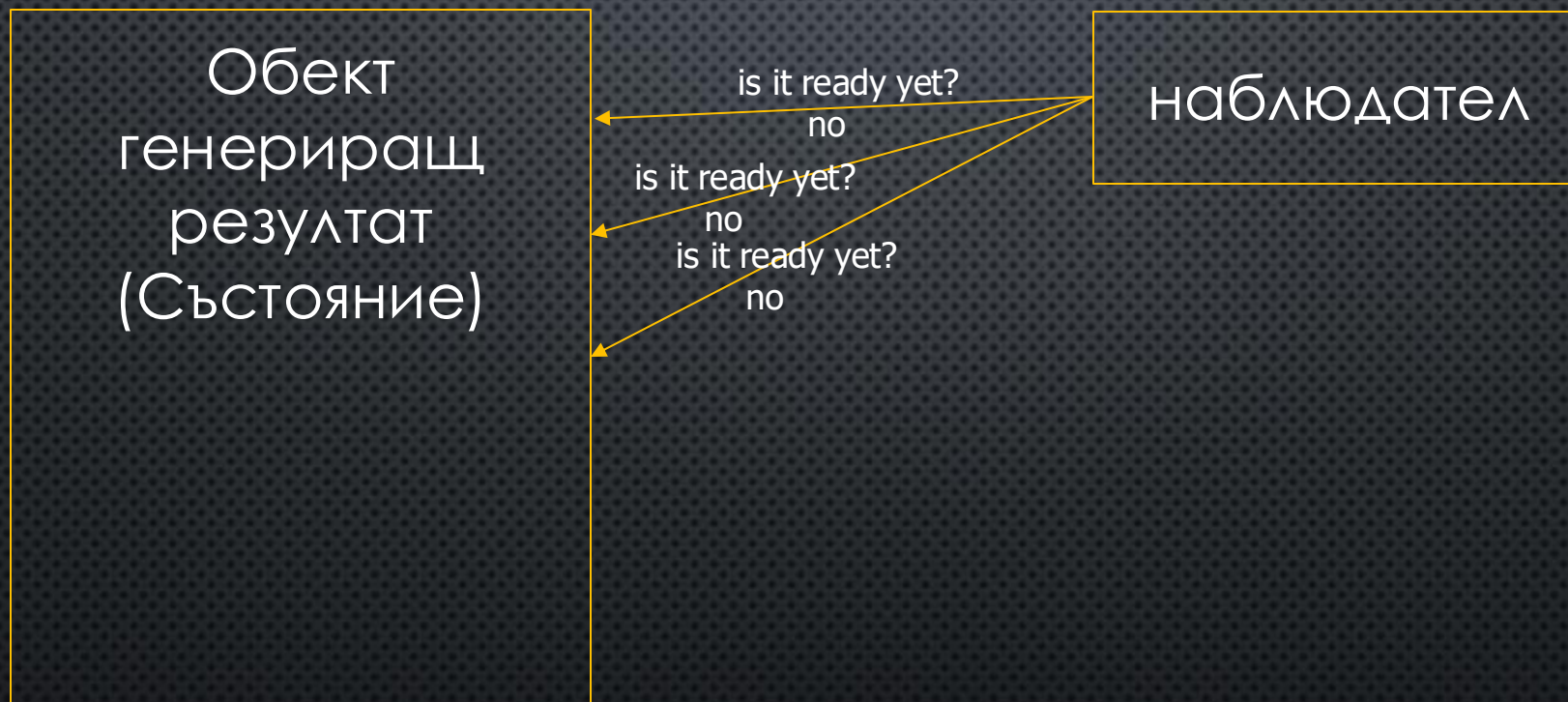
ООП 2

ПОВЕДЕНЧЕСКИ ШАБЛОНИ ЗА ДИЗАЙН

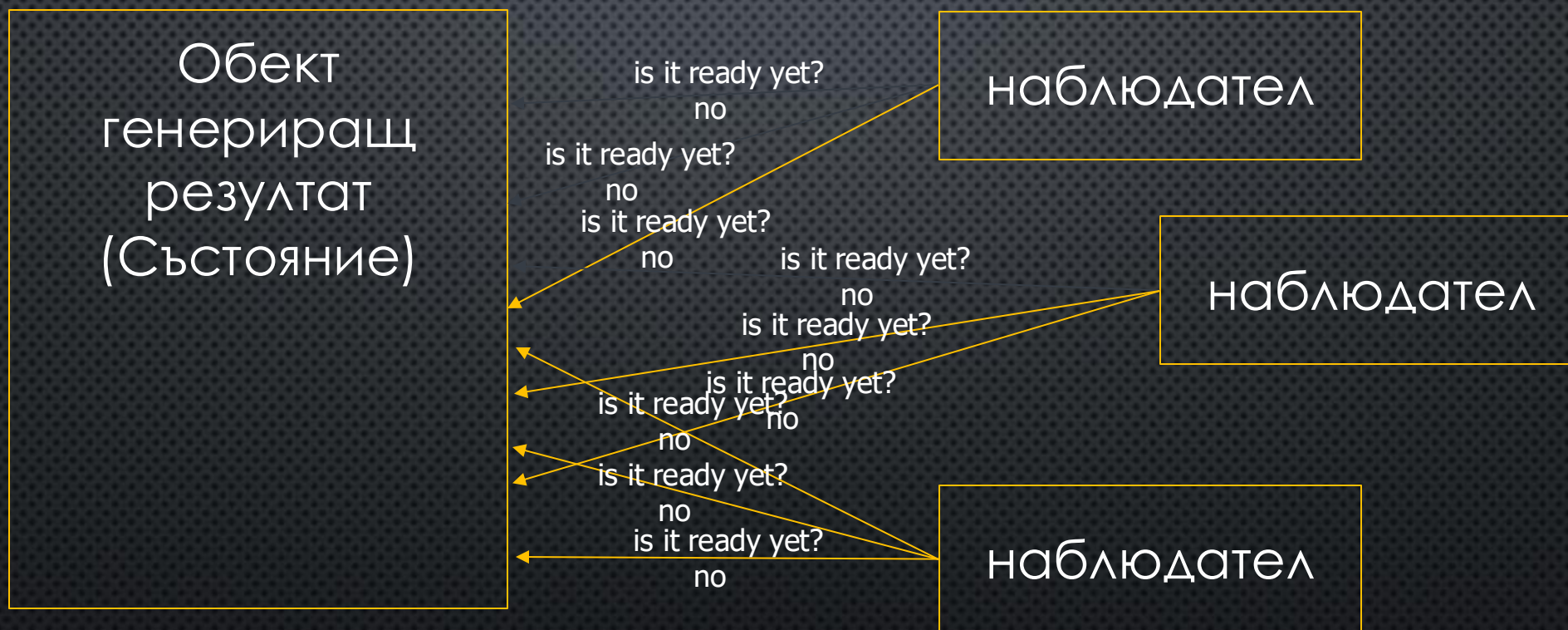
СЪЩНОСТ

- АЛГОРИТМИ И РАЗПРЕДЕЛЯНЕ НА ОТГОВОРНОСТИ МЕЖДУ ОБЕКТИ
 - МОДЕЛИ НА КОМУНИКАЦИЯ МЕЖДУ ОБЕКТИ
- ПОВЕДЕНЧЕСКИ МОДЕЛИ НА НИВО КЛАСОВЕ — НАСЛЕДЯВАНЕ
- ПОВЕДЕНЧЕСКИ МОДЕЛИ НА НИВО ОБЕКТИ - КОМПОЗИЦИЯ

ПРОБЛЕМ



ПРОБЛЕМ



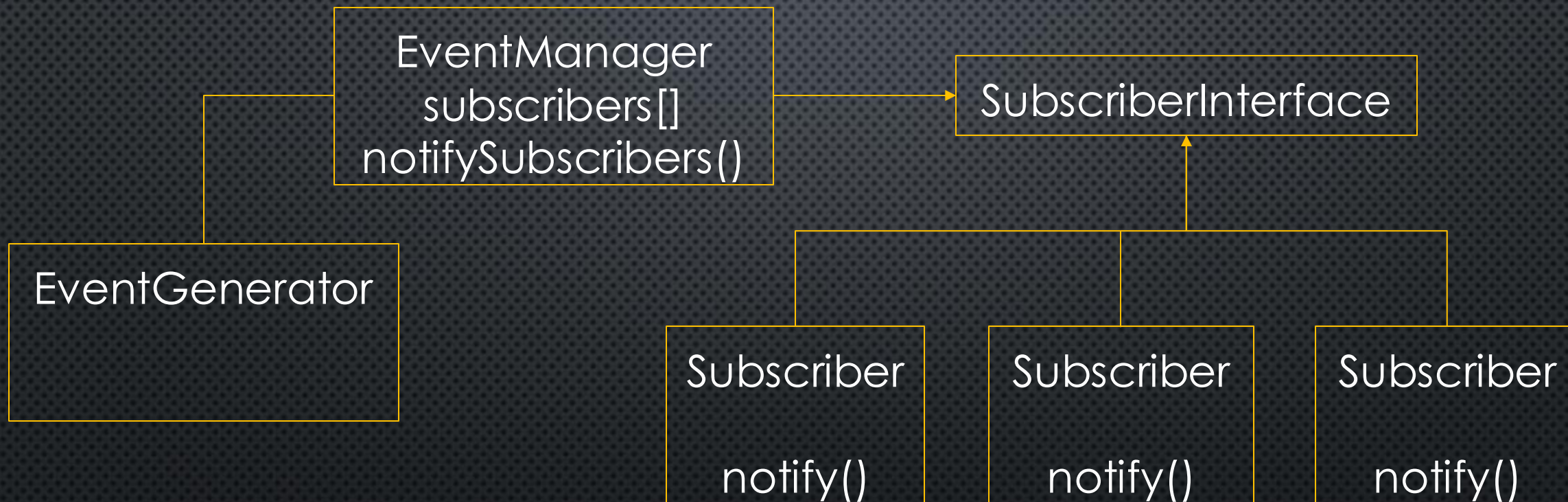
ПРОБЛЕМ



ШАБЛОН НАБЛЮДАТЕЛ (OBSERVER PATTERN)

- ДЕФИНИРА ЗАВИСИМОСТ „ЕДИН КЪМ МНОГО“ МЕЖДУ ОБЕКТИ
 - КОГАТО ЕДИН ОБЕКТ ПРОМЕНИ СЪСТОЯНИЕТО, ВСИЧКИ НЕГОВИ ЗАВИСИМИ СЕ УВЕДОМЯВАТ И АКТУАЛИЗИРАТ АВТОМАТИЧНО
- ОБЕКТ НА НАБЛЮДЕНИЕ (**СУБЕКТ**), НАБЛЮДАТЕЛ
 - АБОНАМЕНТ

ШАБЛОН НАБЛЮДАТЕЛ (OBSERVER PATTERN)

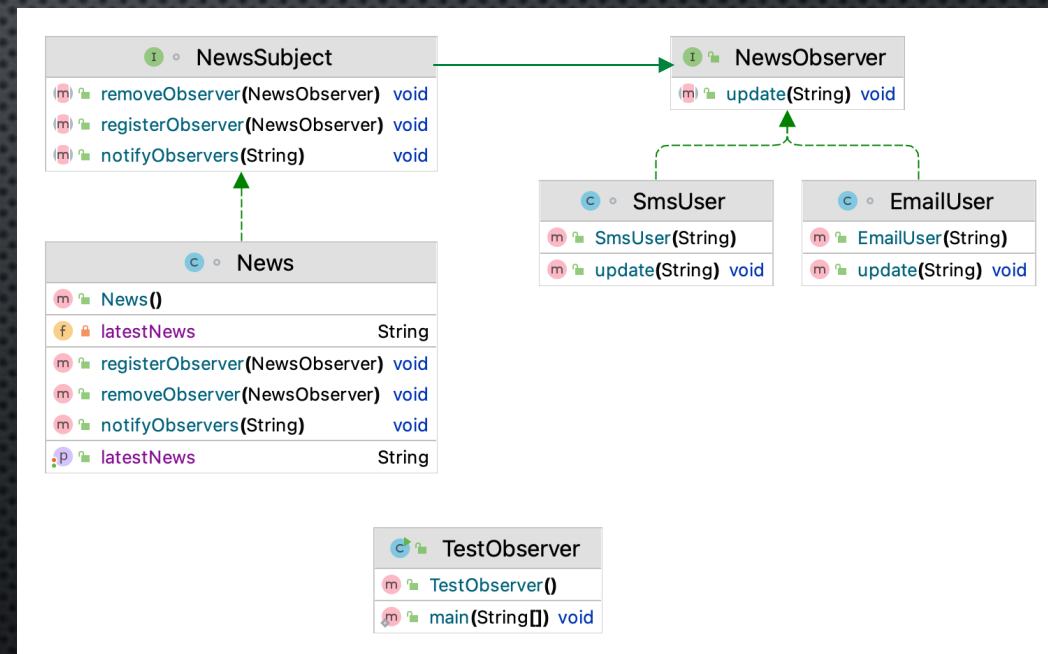


ЗАДАЧА

- КАНАЛ ЗА НОВИНИ
- ПОТРЕБИТЕЛИ
 - EMAIL ПОТРЕБИТЕЛИ
 - SMS ПОТРЕБИТЕЛИ

РЕШЕНИЕ

- „NEWSUBJECT“ – ДЕФИНИРА ПОВЕДЕНИЕ ЗА ВЗАИМОДЕЙСТВИЕ С НАБЛЮДАТЕЛ
- “NEWSOBSERVER” – ДЕФИНИРА ИНТЕРФЕЙС ЗА АКТУАЛИЗИРАНЕ НА ОБЕКТИ, КОИТО ТРЯБВА ДА БЪДАТ УВЕДОМЯВАНИ ЗА ПРОМЕНИ В СУБЕКТА
- “NEWS” – КОНКРЕТЕН СУБЕКТ
- “SMSUSER”, “EMAILUSER” КОНКРЕТНИ НАБЛЮДАТЕЛИ



ШАБЛОН НАБЛЮДАТЕЛЯ ИМПЛЕМЕНТАЦИЯ. OBSERVABLE INTERFACE

```
INTERFACE NEWSUBJECT {  
    VOID REGISTEROBSERVER(NEWSOBSERVER OBSERVER);  
    VOID REMOVEOBSERVER(NEWSOBSERVER OBSERVER);  
    VOID NOTIFYOBSERVERS(STRING NEWS);  
}
```

ШАБЛОН НАБЛЮДАТЕЛ ИМПЛЕМЕНТАЦИЯ. КОНКРЕТЕН СУБЕКТ

```
CLASS NEWS IMPLEMENTS NEWS SUBJECT {  
    PRIVATE LIST<NEWSOBSERVER> OBSERVERS = NEW ARRAYLIST<>();  
    PRIVATE STRING LATESTNEWS;  
  
    PUBLIC STRING GETLATESTNEWS() {  
        RETURN LATESTNEWS;  
    }  
  
    PUBLIC VOID SETLATESTNEWS(STRING LATESTNEWS) {  
        THIS.LATESTNEWS = LATESTNEWS;  
        NOTIFYOBSERVERS(LATESTNEWS);  
    }  
}
```

```
PUBLIC VOID REGISTEROBSERVER(NEWSOBSERVER OBSERVER) {  
    OBSERVERS.ADD(OBSERVER);  
}  
  
PUBLIC VOID REMOVEOBSERVER(NEWSOBSERVER OBSERVER) {  
    OBSERVERS.REMOVE(OBSERVER);  
}  
  
PUBLIC VOID NOTIFYOBSERVERS(STRING NEWS) {  
    FOR (NEWSOBSERVER OBSERVER : OBSERVERS) {  
        OBSERVER.UPDATE(NEWS);  
    }  
}  
}
```

ШАБЛОН НАБЛЮДАТЕЛ ИМПЛЕМЕНТАЦИЯ. OBSERVER INTERFACE

```
PUBLIC INTERFACE NEWSOBSERVER {  
    VOID UPDATE(String NEWS);  
}
```

ШАБЛОН НАБЛЮДАТЕЛ ИМПЛЕМЕНТАЦИЯ. КОНКРЕТНИ НАБЛЮДАТЕЛИ

```
CLASS EMAILUSER IMPLEMENTS NEWSOBSERVER {  
    PRIVATE STRING EMAILADDRESS;  
  
    PUBLIC EMAILUSER(STRING EMAILADDRESS) {  
        THIS.EMAILADDRESS = EMAILADDRESS;  
    }  
  
    PUBLIC VOID UPDATE(STRING NEWS) {  
        SYSTEM.OUT.PRINTLN("SENDING EMAIL TO " + EMAILADDRESS + " WITH NEWS: " + NEWS);  
    }  
}
```

ШАБЛОН НАБЛЮДАТЕЛ ИМПЛЕМЕНТАЦИЯ. КОНКРЕТНИ НАБЛЮДАТЕЛИ

```
CLASS SMSUser IMPLEMENTS NEWSObserver {  
    PRIVATE STRING PHONENUMBER;  
  
    PUBLIC SMSUser(STRING PHONENUMBER) {  
        THIS.PHONENUMBER = PHONENUMBER;  
    }  
  
    PUBLIC VOID UPDATE(STRING NEWS) {  
        SYSTEM.OUT.PRINTLN("SENDING SMS TO " + PHONENUMBER + " WITH NEWS: " + NEWS);  
    }  
}
```

ШАБЛОН НАБЛЮДАТЕЛ ИМПЛЕМЕНТАЦИЯ

```
PUBLIC CLASS TESTOBSERVER {  
    PUBLIC STATIC VOID MAIN(STRING[] ARGS) {  
  
        NEWS NEWS = NEW NEWS();  
        NEWSOBSERVER EMAILUSER1 = NEW EMAILUSER("USER1@EXAMPLE.COM");  
        NEWSOBSERVER SMSUSER1 = NEW SMSUSER("1234567890");  
        NEWSOBSERVER SMSUSER2 = NEW SMSUSER("0987654321");  
        NEWS.REGISTEROBSERVER(EMAILUSER1);  
        NEWS.REGISTEROBSERVER(SMSUSER1);  
        NEWS.REGISTEROBSERVER(SMSUSER2);  
        NEWS.SETLATESTNEWS("NEW PRODUCT LAUNCH!");  
        NEWS.REMOVEOBSERVER(SMSUSER2);  
        NEWS.SETLATESTNEWS("DISCOUNTS AVAILABLE!");  
    }  
}
```

ШАБЛОН НАБЛЮДАТЕЛ ИМПЛЕМЕНТАЦИЯ

SENDING EMAIL TO USER1@EXAMPLE.COM WITH NEWS: NEW PRODUCT LAUNCH!

SENDING SMS TO 1234567890 WITH NEWS: NEW PRODUCT LAUNCH!

SENDING SMS TO 0987654321 WITH NEWS: NEW PRODUCT LAUNCH!

SENDING EMAIL TO USER1@EXAMPLE.COM WITH NEWS: DISCOUNTS AVAILABLE!

SENDING SMS TO 1234567890 WITH NEWS: DISCOUNTS AVAILABLE!

ВГРАДЕН JAVA ШАБЛОН НАБЛЮДАТЕЛ

- JAVA.UTIL
 - OBSERVABLE
 - `ADD_OBSERVER(OBSERVER O), DELETE_OBSERVER(OBSERVER O), DELETE_OBSERVERS(), HAS_CHANGED(), NOTIFY_OBSERVERS(), NOTIFY_OBSERVERS(OBJECT ARG), SET_CHANGED()`
 - OBSERVER
 - `UPDATE(OBSERVABLE O, OBJECT ARG)`

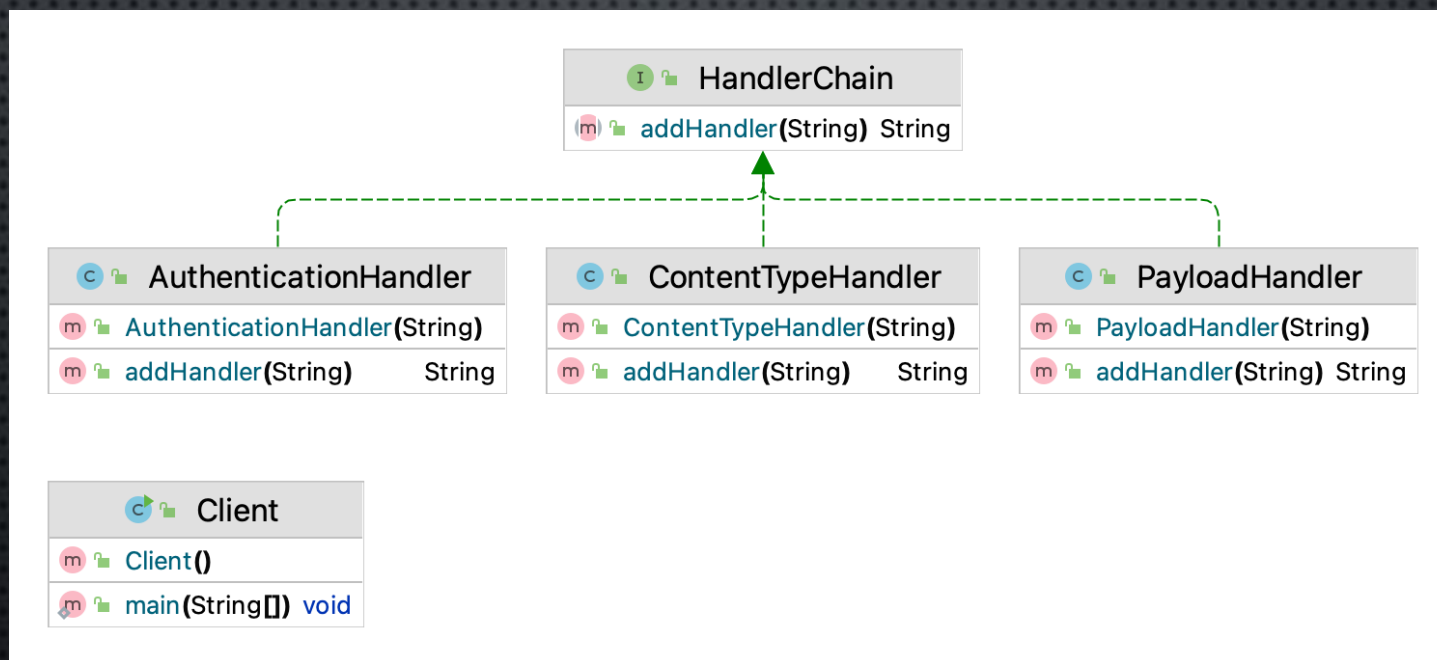
ШАБЛОН НАБЛЮДАТЕЛ ИЗПОЛЗВАНЕ

- КОГАТО ПРОМЯНАТА НА ЕДИН ОБЕКТ ИЗИСКВА ПРОМЯНА НА ДРУГИ
 - БЕЗ ИНФОРМАЦИЯ ЗА БРОЯ ОБЕКТИ КОИТО ТРЯБВА ДА СЕ УВЕДОМЯТ

ШАБЛОН ВЕРИГА НА ОТГОВОРНОСТИ (CHAIN OF RESPONSIBILITY)

- ПРОБЛЕМ!
 - ЗАЯВКА ОТ КЛИЕНТ
 - ОБРАБОТКА
 - РАЗЛИЧНИ ЛОГИКИ
- РЕШЕНИЕ
 - ВЕРИГА ОТ ОБРАБОТЧИЦИ

ШАБЛОН ВЕРИГА НА ОТГОВОРНОСТИ (CHAIN OF RESPONSIBILITY)



ШАБЛОН ВЕРИГА НА ОТГОВОРНОСТИ ИМПЛЕМЕНТАЦИЯ

```
PUBLIC INTERFACE HANDLERCHAIN {  
    STRING ADDHANDLER(STRING INPUTHEADER);  
}
```

ШАБЛОН ВЕРИГА НА ОТГОВОРНОСТИ ИМПЛЕМЕНТАЦИЯ

```
PUBLIC CLASS AUTHENTICATIONHANDLER IMPLEMENTS  
HANDLERCHAIN {
```

```
    STRING TOKEN;
```

```
    PUBLIC HANDLERCHAIN NEXT;
```

```
    PUBLIC AUTHENTICATIONHANDLER(STRING TOKEN) {
```

```
        THIS.TOKEN = TOKEN;
```

```
    }
```

```
    @OVERRIDE
```

```
    PUBLIC STRING ADDHANDLER(STRING INPUTHEADER) {
```

```
        STRING OUTPUTHEADER = INPUTHEADER + "\nAUTHENTICATION: "  
        + TOKEN;
```

```
        IF (NEXT == NULL)
```

```
            RETURN OUTPUTHEADER;
```

```
        ELSE
```

```
            RETURN NEXT.ADDHANDLER(OUTPUTHEADER);
```

```
    }
```

```
}
```

ШАБЛОН ВЕРИГА НА ОТГОВОРНОСТИ ИМПЛЕМЕНТАЦИЯ

```
PUBLIC CLASS CONTENTTYPEHANDLER IMPLEMENTS HANDLERCHAIN {  
  
    STRING CONTENTTYPE;  
    PUBLIC HANDLERCHAIN NEXT;  
  
    PUBLIC CONTENTTYPEHANDLER(STRING CONTENTTYPE) {  
        THIS.CONTENTTYPE = CONTENTTYPE;  
    }  
  
    @OVERRIDE  
    PUBLIC STRING ADDHANDLER(STRING INPUTHEADER) {  
        STRING OUTPUTHEADER = INPUTHEADER + "\nCONTENTTYPE: " +  
            CONTENTTYPE;  
        IF (NEXT == NULL)  
            RETURN OUTPUTHEADER;  
        ELSE  
            RETURN NEXT.ADDHANDLER(OUTPUTHEADER);  
        }  
    }  
}
```

ШАБЛОН ВЕРИГА НА ОТГОВОРНОСТИ ИМПЛЕМЕНТАЦИЯ

```
PUBLIC CLASS PAYLOADHANDLER IMPLEMENTS HANDLERCHAIN {
```

```
    STRING PAYLOAD;
```

```
    PUBLIC HANDLERCHAIN NEXT;
```

```
    PUBLIC PAYLOADHANDLER(STRING PAYLOAD) {
```

```
        THIS.PAYLOAD = PAYLOAD;
```

```
    }
```

```
    @OVERRIDE
```

```
    PUBLIC STRING ADDHANDLER(STRING INPUTHEADER) {
```

```
        STRING OUTPUTHEADER = INPUTHEADER + "\n" + PAYLOAD;
```

```
        IF (NEXT == NULL)
```

```
            RETURN OUTPUTHEADER;
```

```
        ELSE
```

```
            RETURN NEXT.ADDHANDLER(OUTPUTHEADER);
```

```
    }
```

```
}
```

ШАБЛОН ВЕРИГА НА ОТГОВОРНОСТИ ИМПЛЕМЕНТАЦИЯ

```
PUBLIC CLASS CLIENT {  
    PUBLIC STATIC VOID MAIN(STRING[] ARGS) {  
        AUTHENTICATIONHANDLER AUTH = NEW AUTHENTICATIONHANDLER("12345");  
        CONTENTTYPEHANDLER TYPE = NEW CONTENTTYPEHANDLER("JSON");  
        PAYLOADHANDLER PAYLOAD = NEW PAYLOADHANDLER("BODY: {\\"USERNAME\\" = \\"JOHN\\"}");  
  
        AUTH.NEXT = TYPE;  
  
        TYPE.NEXT = PAYLOAD;  
  
        STRING WITHAUTHENTICATION = AUTH.ADDHANDLER("HEADERS WITH AUTHENTICATION");  
        SYSTEM.OUT.PRINTLN(WITHAUTHENTICATION);  
  
        SYSTEM.OUT.PRINTLN("");  
  
        STRING WITHOUTAUTHENTICATION = TYPE.ADDHANDLER("HEADERS WITHOUT AUTHENTICATION");  
        SYSTEM.OUT.PRINTLN(WITHOUTAUTHENTICATION);  
    }  
}
```

ШАБЛОН ВЕРИГА НА ОТГОВОРНОСТИ ИМПЛЕМЕНТАЦИЯ

HEADERS WITH AUTHENTICATION

AUTHENTICATION: 12345

CONTENTTYPE: JSON

BODY: {"USERNAME" = "JOHN"}

HEADERS WITHOUT AUTHENTICATION

CONTENTTYPE: JSON

BODY: {"USERNAME" = "JOHN"}